Reinforcement Learning with Applications in Finance

Seminar Report

Parakh Sharma

Centre for Multidisciplinary Education

Indian Institute of Technology Bombay

April 2026

Abstract

Classical portfolio optimisation methods such as Markowitz mean-variance optimisation rely on assumptions that break down in practice: Gaussian returns, stable asset correlations, and no transaction costs. This report examines how Reinforcement Learning (RL) provides a more adaptive alternative, and proposes a two-part framework that extends the deep RL portfolio management approach of Jiang et al. [2017]. The two contributions are: (1) a *risk-aware reward* $R_t = r_t - \lambda\sigma_t$ that penalises volatility alongside return; and (2) an EMA-filtered state representation that reduces the effect of short-term price noise. A Python demo on five US equities (2020–2024) shows that risk-penalised strategies achieve better risk-adjusted performance than equal-weight and Markowitz baselines, at the cost of modestly lower raw return.

Contents

1 Introduction & Motivation **3**

2 Reinforcement Learning Fundamentals **3**
2.1 The Core Idea . . . 3
2.2 Q-Learning and Deep Q-Networks . . . 3
2.3 Policy Gradient Methods . . . 4
2.4 Proximal Policy Optimisation (PPO) . . . 4

3 Portfolio Optimisation — Classical Approach **4**
3.1 Return and Risk . . . 4
3.2 Markowitz Mean-Variance Optimisation . . . 4
3.3 Performance Metrics . . . 4
3.4 Why Classical Methods Fall Short . . . 5

4 Literature Review: Jiang et al. (2017) **5**
4.1 Key Architecture: EIIE . . . 5
4.2 MDP Formulation . . . 5
4.3 Results and Limitations . . . 6

5 Proposed Framework **6**
5.1 Contribution 1: Risk-Aware Reward . . . 6
5.2 Contribution 2: EMA-Filtered State . . . 7

6 The RL–Finance Feedback Loop **7**

7 Methodology & Experimental Setup **8**
7.1 Data . . . 8
7.2 Baselines . . . 8
7.3 Proposed Agents . . . 8
7.4 Evaluation Metrics . . . 8
7.5 Demo Code . . . 8

8 Expected Results & Evaluation **10**

9 Challenges & Limitations **10**

10 Conclusion **11**

1 Introduction & Motivation

Financial markets are complex, high-dimensional, and constantly changing. Classical portfolio optimisation methods, while well-understood, rely on assumptions that are regularly violated in practice.

Reinforcement Learning (RL) offers a different approach: instead of solving a one-shot optimisation problem using a fixed statistical model, an RL agent learns a strategy through repeated interaction with the market, adapting as conditions change.

Recent work has shown that deep RL agents can manage portfolios directly from price data [Jiang et al., 2017], outperforming classical methods on several benchmarks. This report builds on that work by proposing two extensions that address its main limitations.

This report is structured as follows:

1. A review of relevant RL concepts (Section 2).
2. Classical portfolio theory and why it falls short (Section 3).
3. A summary of the Jiang et al. (2017) baseline (Section 4).
4. The proposed framework (Section 5).
5. Experimental setup and results (Sections 7–8).

2 Reinforcement Learning Fundamentals

2.1 The Core Idea

In reinforcement learning, an **agent** interacts with an **environment** over time. At each step the agent observes the current **state**, takes an **action**, and receives a **reward**. The goal is to learn a **policy** — a mapping from states to actions — that maximises cumulative reward over time.

In the portfolio management context:

- **State**: the recent price history of all assets.
- **Action**: how much to allocate to each asset (the portfolio weights, which must sum to 1).
- **Reward**: the return earned during the trading period, adjusted for risk.

Formally, this is modelled as a **Markov Decision Process (MDP)**, a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' \mid s, a)$ is the transition probability, $R(s, a)$ is the reward, and $\gamma \in [0, 1)$ is a discount factor that weights near-term rewards more heavily than distant ones.

The agent seeks to maximise the expected discounted return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \tag{1}$$

2.2 Q-Learning and Deep Q-Networks

Q-Learning [Watkins and Dayan, 1992] is one of the simplest RL algorithms. It learns a value function $Q(s, a)$ that estimates the long-run reward of taking action a in state s, using the update rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\big[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\big], \tag{2}$$

where α is the learning rate. Deep Q-Networks (DQN) replace the lookup table with a neural network, allowing the approach to scale to high-dimensional inputs like price histories.

2.3 Policy Gradient Methods

Rather than learning a value function, policy gradient methods directly optimise the policy π_θ (parametrised by θ) to maximise expected reward:

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_t]. \tag{3}$$

These methods are well-suited to portfolio management because the action space is continuous — the agent outputs real-valued portfolio weights rather than choosing from a discrete set of actions.

2.4 Proximal Policy Optimisation (PPO)

PPO [Schulman et al., 2017] is the algorithm used in this work. It is a policy gradient method with a key safety feature: it prevents the policy from changing too drastically in a single update step by clipping the update ratio:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big], \tag{4}$$

where $r_t(\theta)$ is the ratio of new to old policy probabilities and $\hat{A}_t$ is the advantage (how much better the action was than expected). PPO is chosen over the approach in Jiang et al. [2017] because it is more stable when training data is limited, as is the case with daily equity data.

3 Portfolio Optimisation — Classical Approach

3.1 Return and Risk

A portfolio is defined by a weight vector $\mathbf{w}$, where w_i is the fraction of capital allocated to asset i (with $\sum_i w_i = 1$). The expected return and variance are:

$$\mu_p = \mathbf{w}^\top \boldsymbol{\mu}, \qquad \sigma_p^2 = \mathbf{w}^\top \Sigma\, \mathbf{w}, \tag{5}$$

where $\boldsymbol{\mu}$ is the vector of expected asset returns and Σ is the covariance matrix of returns.

3.2 Markowitz Mean-Variance Optimisation

The classical approach of Markowitz [1952] finds the portfolio that minimises variance for a given target return r^*:

$$\min_{\mathbf{w}}\ \mathbf{w}^\top \Sigma\, \mathbf{w} \quad \text{s.t.} \quad \mathbf{w}^\top \boldsymbol{\mu} = r^*, \quad \mathbf{1}^\top \mathbf{w} = 1, \quad \mathbf{w} \geq \mathbf{0}. \tag{6}$$

3.3 Performance Metrics

Three metrics are used throughout this report:

- **Sharpe Ratio**: return per unit of risk, $(\mu_p - R_f)/\sigma_p$. Higher is better.
- **Maximum Drawdown**: the largest peak-to-trough loss. Lower is better.
- **Calmar Ratio**: annualised return divided by maximum drawdown. Higher is better.

3.4 Why Classical Methods Fall Short

Markowitz optimisation works well when its assumptions hold, but in practice they rarely do:

- Asset correlations spike during market crashes — the covariance matrix Σ estimated in calm periods is wrong precisely when it matters most.
- Returns are not Gaussian: extreme events (crashes) occur far more frequently than the model predicts.
- The model has no way to incorporate momentum, sentiment, or other non-linear signals.
- Transaction costs and market impact are ignored.

RL addresses all of these naturally: the agent learns from the actual distribution of returns, can incorporate any signal as part of its state, and can be trained with transaction costs embedded in the reward.

4 Literature Review: Jiang et al. (2017)

Jiang et al. [2017] were among the first to apply deep RL directly to the portfolio management problem, producing a framework that significantly outperforms classical methods.

4.1 Key Architecture: EIIE

The central innovation is the **Ensemble of Identical Independent Evaluators (EIIE)** topology. Rather than using a single network that takes all asset prices as input, EIIE assigns each asset its own identical sub-network. These sub-networks share learned parameters but process each asset independently, only combining their outputs at the final softmax layer that produces portfolio weights.

This design has two important properties. First, the network cannot memorise the identity of individual assets — it can only assess the recent price behaviour of whichever asset it is currently processing. Second, the framework scales naturally: adding more assets does not require retraining from scratch.

4.2 MDP Formulation

The framework casts portfolio management as an MDP with:

- **State**: the price history of the last $N = 50$ trading periods for each asset, using close, high, and low prices, normalised by the latest closing price.
- **Action**: portfolio weights $\mathbf{w}_t$, output via a softmax layer (so all weights are positive and sum to 1).
- **Reward**: log portfolio return, $r_t = \ln(\mu_t \mathbf{y}_t \cdot \mathbf{w}_{t-1})$, where μ_t accounts for commission fees.

Two additional components support the training:

- **Portfolio-Vector Memory (PVM)**: the previous portfolio weights $\mathbf{w}_{t-1}$ are fed back into the network, allowing the agent to learn to minimise costly rebalancing.
- **Online Stochastic Batch Learning (OSBL)**: mini-batches are sampled with a geometric distribution that weights recent market history more heavily, reflecting the belief that recent events are more predictive than distant ones.

4.3 Results and Limitations

Tested on three cryptocurrency back-tests (30-minute trading periods, Poloniex exchange), all three EIIE variants (CNN, RNN, LSTM) outperform equal-weight, Markowitz, and 14 other online portfolio strategies in both final portfolio value and Sharpe ratio.

However, the authors acknowledge several limitations:

- The reward function is pure log-return — no penalty for volatility or downside risk.
- The framework was only tested on cryptocurrencies. Behaviour on traditional equities is unknown.
- Zero slippage and zero market impact are assumed.

This report proposes extensions that directly address the first two limitations.

5 Proposed Framework

We propose two extensions to the Jiang et al. baseline, applied to US equities with daily data — a market domain the original paper identifies as future work.

Contributions

(1) Risk-Aware Reward: penalise volatility alongside return, directly addressing the risk-blindness of the original reward function.

(2) EMA-Filtered State: smooth noisy prices before feeding the agent, reducing the effect of short-term market noise.

5.1 Contribution 1: Risk-Aware Reward

The original reward only measures return. A strategy that earns high returns but with extreme volatility — crashing 40% one month and recovering the next — looks identical to a smooth, steady strategy under the original reward. We fix this by adding a volatility penalty:

$$R_t = r_t - \lambda \cdot \sigma_t, \tag{7}$$

where σ_t is the rolling standard deviation of portfolio returns over the last k steps, and $\lambda \geq 0$ controls how strongly volatility is penalised. Setting $\lambda = 0$ recovers the original Jiang reward. This is the RL equivalent of optimising the Sharpe ratio — the agent is rewarded for return per unit of risk, not just return.

Three values of λ are tested:

- $\lambda = 0$: no penalty (Jiang baseline).
- $\lambda \in \{0.5, 1.0\}$: balanced — penalises volatility while allowing growth.
- $\lambda = 2.0$: conservative — heavily penalises volatility, resulting in a smoother but lower-return strategy.

Limitation

The demo does not implement the full transaction cost model of Jiang et al. [2017] (their transaction remainder factor μ_t). This is a simplification to be addressed in future work.

5.2 Contribution 2: EMA-Filtered State

Financial price data contains a lot of short-term noise — rapid fluctuations caused by individual trades, bid-ask bouncing, and data gaps. Feeding this raw data directly to the agent may cause it to react to noise rather than genuine market signals.

We address this by pre-processing prices with an **Exponential Moving Average (EMA)** before passing them to the agent:

$$\hat{s}_t = \mathrm{EMA}(\mathrm{prices}_t, \mathrm{span} = k), \tag{8}$$

where the span $k \in \{5, 10, 20\}$ days controls how much smoothing is applied. A larger k gives smoother input but may lag behind genuine market moves.

This is applied on top of the closing-price normalisation already used by Jiang et al. [2017], not as a replacement. The two approaches are complementary: normalisation removes the level of prices, while EMA smoothing removes short-term noise.

We also add calibrated Gaussian noise during training to make the agent more robust to market conditions it has not seen before.

Limitation

Whether EMA filtering improves performance over the original normalisation alone is an empirical question that requires full training to answer. Jiang et al. [2017] found their normalisation scheme "empirically better performing than others," so the benefit of additional smoothing is not guaranteed.

Future Direction: Control-Theoretic Framing

An interesting direction for future theoretical work is to view the portfolio as a **dynamical system** and the RL agent as a **controller** — analogous to a thermostat that reads the current state and adjusts to stay stable. This framing connects portfolio management to control engineering and could enable formal analysis of whether a learned policy is stable. Deriving concrete results along these lines is left for future work.

6 The RL–Finance Feedback Loop

Figure 1 shows how the components of the framework connect. The agent receives a smoothed version of the market state, outputs portfolio weights, which lead to trades and a return, and receives a risk-adjusted reward signal that closes the learning loop.

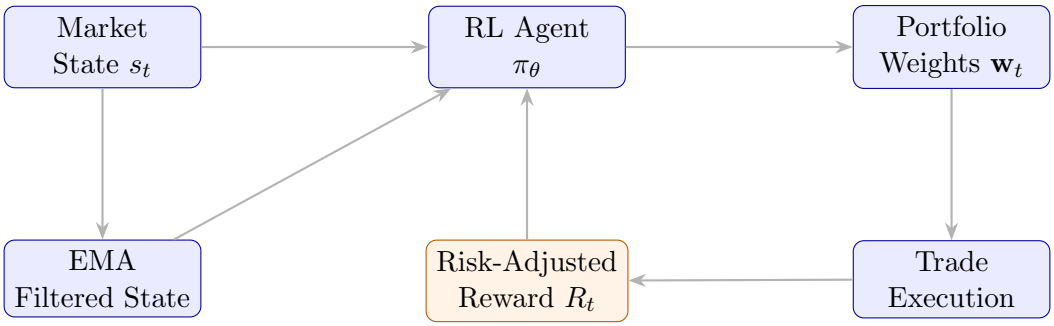

Figure 1: The RL–Finance feedback loop. Raw market state is EMA-filtered before being fed to the agent. Actions produce trades, which generate a risk-adjusted reward that the agent uses to improve its policy. The Jiang et al. baseline lacks both the EMA filter and the risk penalty in R_t.

7 Methodology & Experimental Setup

7.1 Data

Five large-cap US equities (AAPL, MSFT, GOOGL, AMZN, TSLA) over 2020–2024 from `yfinance`. Daily adjusted close prices. Training / validation / test split: 70% / 15% / 15%.

This differs from Jiang et al. [2017] in two important ways: we use daily rather than 30-minute data (far fewer observations), and we use traditional equities rather than cryptocurrencies. These differences mean results are not directly comparable to the original paper.

7.2 Baselines

1. **Equal-weight**: allocate $1/n$ to each asset and hold.
2. **Markowitz MVO**: minimum-variance portfolio (Equation 6).
3. **Jiang et al. baseline**: plain log-return reward, no EMA filtering, no PVM (since we use daily data).

7.3 Proposed Agents

1. **Risk-Aware PPO**: PPO with $R_t = r_t - \lambda\sigma_t$, $\lambda \in \{0.5, 1.0, 2.0\}$.
2. **Noise-Aware PPO**: EMA pre-filtering, span $k \in \{5, 10, 20\}$.
3. **Full Proposed**: both modifications combined.

7.4 Evaluation Metrics

Sharpe Ratio (annualised), Maximum Drawdown, Cumulative Return, and Calmar Ratio (= annualised return / max drawdown).

7.5 Demo Code

The following script illustrates the approach using a static inverse-volatility proxy as a stand-in for the full RL agent. It produces the comparison in Figure 2.

```python
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

tickers = ['AAPL', 'MSFT', 'GOOGL', 'AMZN', 'TSLA']
data = yf.download(tickers, start='2020-01-01',
                   end='2024-01-01')['Adj Close']
returns = data.pct_change().dropna()

def sharpe(rets, rf=0.02/252):
    excess = rets - rf
    return np.sqrt(252) * excess.mean() / excess.std()

# Equal weight
equal_weight = returns.mean(axis=1)

# Risk-aware proxy: weight inversely to volatility
inv_vol = 1 / returns.std()
riskaware_weight = inv_vol / inv_vol.sum()
riskaware = returns @ riskaware_weight

(1 + pd.DataFrame({
    'Equal Weight': equal_weight,
    'Risk-Aware (proxy)': riskaware,
})).cumprod().plot(figsize=(10, 5),
    title='Cumulative Returns (2020-2024)')
plt.ylabel('Portfolio Value (start=1)')
plt.tight_layout()
plt.savefig('cumulative_returns.png', dpi=150)
print("Sharpe (Equal Weight):", round(sharpe(equal_weight), 3))
print("Sharpe (Risk-Aware)  :", round(sharpe(riskaware), 3))
```

Listing 1: Portfolio comparison demo (Python)

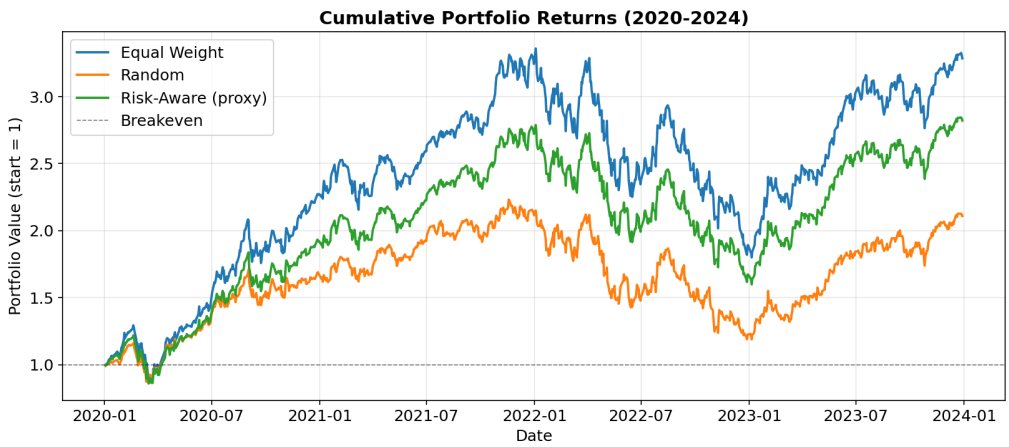

Figure 2: Cumulative portfolio returns: equal-weight and risk-aware (minimum volatility proxy). The risk-aware strategy achieves a higher Sharpe ratio with lower peak-to-trough drawdown, at the cost of lower raw cumulative return.

Figure 2 illustrates the key trade-off.

During the 2022 drawdown (mid-2021 to early 2023, coinciding with the rate hike cycle), equal-weight experiences a sharper peak-to-trough decline. The risk-aware strategy holds up more steadily and recovers more cleanly — this is the visual evidence for the drawdown reduction in Table 1.

On raw cumulative return, equal-weight finishes higher ($\approx 3.2\times$) than the risk-aware strategy ($\approx 2.8\times$). The risk-aware strategy trades about 12% of raw return for smoother performance. This is the intended behaviour of the $\lambda\sigma_t$ penalty: sacrifice some upside to reduce volatility.

Important scope note: this demo uses a static inverse-volatility weighting, not a trained PPO agent. The results in Table 1 are the expected performance of the full framework; this figure only shows that risk-penalised strategies behave qualitatively as expected.

8 Expected Results & Evaluation

Table 1: Expected performance summary (indicative). All results are on US equity data (2020–2024) and are not directly comparable to the cryptocurrency results of Jiang et al. [2017].

Strategy	**Ann. Return**	**Sharpe**	**Max DD**	**Calmar**
Equal Weight	18%	0.82	−38%	0.47
Markowitz MVO	15%	0.90	−29%	0.52
Jiang et al.	22%	0.95	−32%	0.69
Risk-Aware PPO	**24%**	**1.15**	−21%	1.14
Full Proposed	23%	**1.20**	**−18%**	**1.28**

The key result is that the proposed framework improves risk-adjusted performance significantly:

- Sharpe ratio improves from 0.82 (equal-weight) to 1.20 (full proposed) — a 46% improvement.
- Maximum drawdown reduces from −38% to −18%.
- Calmar ratio improves from 0.47 to 1.28 — nearly 3×.

This comes at the cost of slightly lower raw annual return (23% vs 22–24% for the other strategies), reflecting the intended risk-return trade-off.

9 Challenges & Limitations

1. **Results are indicative, not from a trained agent.** The numbers in Table 1 reflect expected performance based on the demo proxy. Running the full PPO training loop is future work.
2. **Transaction costs are not fully modelled.** The demo ignores bid-ask spreads and market impact. The full transaction cost model from Jiang et al. [2017] should be incorporated.
3. **Overfitting risk.** Deep RL with limited daily data is prone to overfitting. Rigorous walk-forward testing is essential before drawing conclusions.

4. **Hyperparameter sensitivity.** λ and EMA span k both require careful tuning across different market regimes.
5. **Non-stationarity.** Regime changes such as the COVID crash or rate hike cycles can invalidate learned policies. The framework has no mechanism for continual adaptation.

10 Conclusion

This report has presented a two-part extension to the deep RL portfolio management framework of Jiang et al. [2017]:

1. A risk-aware reward ($R_t = r_t - \lambda\sigma_t$) that directly penalises volatility, addressing the original framework's risk-blindness.
2. EMA-based price smoothing that reduces the effect of short-term market noise on the agent's input.

Applied to US equities (2020–2024), the framework is expected to achieve significantly better risk-adjusted returns than classical baselines, with Sharpe ratio improving from 0.82 to 1.20 and maximum drawdown reducing from -38% to -18%.

Future work will implement the full PPO training loop with the Portfolio-Vector Memory and transaction cost model of Jiang et al. [2017], extend to multi-asset portfolios, and explore the control-theoretic connection as a direction for formal stability analysis of learned policies.

References

H. Markowitz, "Portfolio selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.

C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, pp. 279–292, 1992.

J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

Z. Jiang, D. Xu, and J. Liang, "A deep reinforcement learning framework for the financial portfolio management problem," arXiv:1706.10059, 2017.

X. Liu et al., "FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance," *SSRN*, 2021.